Privilege Containment Design Patterns for Self-Healing LLM Agent Operating Systems

Anonymous Author
Anonymous Affiliation

Abstract

Self-healing LLM agent operating systems increasingly adopt operating-system design principles—agents scheduled as processes, tools gated by permission checkers, and recovery orchestrators that mutate agent topology on failure. A classical OS invariant—*an exception handler executes with the privileges of the faulting context, never higher*—has been enforced in kernels for decades, yet its transfer to the recovery path of LLM agent systems has not been examined. We show that this invariant is systematically violated: when a recovery orchestrator restructures the agent graph (e.g., replacing a failing agent's role), the new role is frequently selected by a diagnostic LLM whose input includes attacker-influenced content embedded in the failure payload, and the role-replacement sink performs no authorization check on the value it consumes. This is an instance of CWE-862 (Missing Authorization) at the recovery sink and the Confused Deputy problem at the orchestrator—two complementary framings of the same structural gap.

We approach this gap as a software architecture contribution, not a security measurement study. We formalize the Privilege-Downgrade Invariant—$P(\text{recovery action}) \leq P(\text{failed execution})$—as the agent-systems analog of the OS exception-handler privilege rule, and propose a design pattern that enforces it at a single centralized choke point: the Reauthorization Gate. The pattern is realized by (i) extending the recovery result type with a `requiresReauthorization` flag that side-effecting strategies set, (ii) deferring the side effect as a callback stored in the recovery context rather than executing it inline, and (iii) intercepting every flagged result at the orchestrator—the single component through which every recovery action must flow—and routing it through the existing `PermissionChecker` before the deferred callback executes. This localization turns the security property "no recovery action exceeds the privilege of the failed execution" from an emergent property of independently-authored strategies into a local, inspectable property of one orchestrator method.

We validate the pattern on Recova, a multi-agent LLM operating system prototype (Java 21, 14 recovery strategy classes, four registered roles). Under the baseline configuration (no gate), a topology-mutation adversary that embeds role-escalation hints in a semantic core dump achieves a perfect 1.00 attack success rate across 100 trials, because the LLM-diagnosed role reaches the replacement sink without authorization. Under the defended configuration, the gate rejects every out-of-whitelist role *by construction*—the invariant

is a programmatic check, not a probabilistic defense—yielding 0.00 attack success, zero false alarms on benign recoveries, and sub-millisecond machinery overhead. Because the gate is LLM-independent, the guarantee is expected to hold across all models regardless of instruction-following strength.

The contribution is framed as a design pattern transfer: classical OS invariants (capability-based authority, exception-handler privilege containment) are ported to a domain—LLM agent recovery—where they have been conspicuously absent. The pattern is framework-agnostic in principle: it depends only on the existence of a recovery orchestrator and a permission checker, not on any specific agent architecture. The empirical validation establishes implementability and cost; a broader empirical evaluation (statistical generalization, cross-framework exploitability, adaptive adversaries) is beyond the scope of this architectural paper.

Keywords: Design patterns, Privilege containment, Self-healing LLM agents, Operating-system principles, CWE-862, Confused deputy, Exception handler privilege invariant

---

1. Introduction

Context: LLM Agent Operating Systems and Self-Healing. Modern LLM agent systems increasingly adopt operating-system design principles: agents are scheduled as processes with per-agent roles, tools are syscalls gated by a permission checker that enforces role-based access control, tenants are isolated, and—crucially—*automated self-healing* is employed to handle the frequent failures inherent to LLM execution Wu et al. (2023); Yang et al. (2024); [Authors] (2026). When an agent fails (verification failure, capability mismatch, or semantic crash), a recovery orchestrator classifies the fault and dispatches a layered chain of recovery strategies: re-injection of error context, model fallback, *topology mutation* (replacing the failing agent with a different role), and human escalation. This architectural pattern—which we call the *recovery pipeline*—is motivated by the empirical observation that retrying with structured diagnostic context, or restructuring the agent graph to better match the task, frequently succeeds where the original attempt did not Shinn et al. (2023); Madaan et al. (2023); Gou et al. (2024).

Problem: Privilege Escalation via Topology Mutation. Despite the richness of this recovery machinery, the privilege invariants governing *recovery actions* have not been systematically examined. The recovery pipeline inherits an implicit assumption from classical OS exception handling: *that the diagnostic information triggering a recovery action is a faithful report of the failure's cause, and that acting on it—including escalating privileges or replacing roles—is safe because the failure is presumed benign.* This assumption holds in classical kernels because exception payloads (e.g., page-fault addresses) are generated by the trusted computing base. It *does not hold* in LLM agent systems, where the failure payload—a semantic core dump capturing the agent's last input and context on crash—frequently embeds *external content that the agent was processing when it crashed.* A diagnostic LLM that reads the core dump may be tricked into emitting an

attacker-suggested high-privilege role, which then reaches the node-replacement primitive without a permission re-check.

From a software architecture perspective, this is a *privilege containment failure at the recovery sink*: the role-replacement action bypasses the very permission checks that the forward execution path enforces. We characterize this failure along two complementary dimensions:

- **CWE-862 (Missing Authorization) at the recovery sink.** The role-replacement primitive consumes a role value influenced by content that originated outside the trusted computing base, yet performs no authorization check on that value MITRE Corporation (2024). This is a decades-old vulnerability class—"access control 101"—but its *systematic appearance* in self-healing recovery paths is an architectural finding about how the recovery pipeline is assumed benign and therefore exempted from forward-path invariants.
- **Confused Deputy at the orchestrator.** The recovery orchestrator holds the privilege to swap agent roles—a capability not granted to ordinary tool calls—and, in the baseline configuration, it exercises this capability on the basis of an LLM diagnosis whose input was supplied by attacker-influenced content. This is a textbook instance of the Confused Deputy problem Hardy (1988): a privileged program is tricked by a less-privileged caller into misusing its authority on the caller's behalf.

The two framings are complementary, not redundant: CWE-862 identifies the missing check at the *sink* (the role-replacement primitive), while the Confused Deputy framing identifies the authority-misuse pattern at the *deputy* (the orchestrator). Both motivate the same structural fix—a capability-based privilege check at a centralized choke point.

Research Questions. This paper addresses three research questions from a software architecture perspective:

RQ1. How can the OS exception-handler privilege invariant—"an exception handler executes with the privileges of the faulting context, never higher"—be formalized and ported to the recovery path of an LLM agent operating system? (§3, §4)

RQ2. What design pattern enforces this invariant at the orchestrator level, and how does it decompose into concrete code-level mechanisms (result-type extension, deferred side effects, centralized interception)? (§4)

RQ3. Is the pattern implementable on a representative multi-agent LLM OS, and at what cost (false alarms, latency overhead)? Does the guarantee hold independently of the underlying LLM? (§5)

Methodology. We employ a design-science methodology Hevner et al. (2004). (i) *Problem formalization*: we identify the privilege containment gap at the recovery sink by mapping the OS exception-handler privilege invariant onto the LLM agent recovery path, and we trace its root cause to the decentralized validation of

recovery actions. (ii) *Pattern design*: we design a centralized enforcement pattern—the REAUTHORIZATION GATE—that ports capability-based authority (the classic Confused Deputy fix Hardy (1988)) to the agent recovery orchestrator. (iii) *Implementation and validation*: we implement the pattern in Recova, a multi-agent LLM operating system prototype (Java 21, 14 recovery strategy classes), and validate its implementability, efficacy, and cost on a controlled testbed.

Contributions. This paper makes the following architectural contributions:

- **Formalization of the Privilege-Downgrade Invariant for LLM agent recovery.** We identify that the OS exception-handler privilege invariant—"P(handler) $\leq$ P(faulting context)"—does not transfer automatically to LLM agent recovery paths, where the diagnostic LLM's output can be influenced by attacker-controlled content embedded in the failure payload. We formalize the agent-systems analog—P(recovery action) $\leq$ P(failed execution)—and trace two complementary root causes: CWE-862 (Missing Authorization) at the recovery sink, and the Confused Deputy pattern at the orchestrator.
- **A centralized design pattern: the Reauthorization Gate.** We propose a design pattern that enforces the invariant at a single orchestrator-level choke point through which every side-effecting recovery action must flow. The pattern decomposes into three concrete mechanisms: (a) a `requiresReauthorization` flag on the recovery result type, (b) deferred side effects stored as callbacks in the recovery context, and (c) orchestrator-level interception that routes flagged results through the existing `PermissionChecker` before any callback executes. This localization turns the security property from an emergent property of independently-authored strategies into a local, inspectable property of one orchestrator method—a classic cross-cutting concern pattern applied to recovery-path privilege enforcement.
- **Validation of implementability, efficacy, and cost.** We implement the pattern in Recova and validate it on a controlled testbed. The baseline configuration (no gate) yields a perfect 1.00 attack success rate for topology-mutation privilege escalation; the defended configuration yields 0.00 *by construction* (the whitelist rejects out-of-set roles), with zero false alarms on benign recoveries and sub-millisecond machinery overhead. Because the gate is a programmatic check on a whitelist, the guarantee is LLM-independent and expected to hold across all models.

Positioning. This is a software architecture and design pattern paper, not a security measurement study. The testbed (Recova) is the authors' own prototype; the contribution is the formalization of the invariant, the design of the centralized enforcement pattern, and the demonstration that the pattern is implementable with negligible cost. A broader empirical evaluation (statistical generalization across production frameworks, cross-model robustness of *content-level* defenses, adaptive adversaries) is beyond the scope of this paper and is addressed by complementary empirical work in the agent-security literature Debenedetti et al. (2025); Zhan et al. (2025).

Data Availability. To facilitate future research and reproducibility, we will open-source the design pattern implementation, the validation harness, and the anonymized validation logs upon publication.

The remainder of this paper is organized as follows. Section 2 positions our work relative to OS privilege management, the Confused Deputy literature, and LLM agent architectures. Section 3 formalizes the privilege containment gap at the recovery sink. Section 4 details the REAUTHORIZATION GATE design pattern. Section 5 validates the pattern on the testbed. Section 6 discusses architectural design recommendations and a migration guide. Section 7 concludes.

2. Related Work

We position our work along three axes: prior studies of security vulnerabilities in LLM agents, the emerging literature on self-healing and autonomous error recovery in agent systems, and the long tradition of privilege management in operating systems. Our contribution sits at the intersection of these three bodies of work: we show that the *recovery sink*—an artifact of self-healing design—constitutes a privilege-containment gap that prior security work has not examined, and we demonstrate that the classical OS exception-handler privilege invariant ports naturally to close it.

2.1. Security Vulnerabilities in LLM Agents

Prompt injection. The canonical vulnerability class for LLM agents is *prompt injection*, in which adversary-controlled text is incorporated into the model's context and subsequently obeyed as if it were a trusted instruction. Greshake et al. introduced *indirect prompt injection*, demonstrating that a fetched web page can embed directives that the agent executes with the privileges of its system prompt Greshake et al. (2023). Willison Willison (2023) catalogued the practical exploitability of injection against production agent frameworks. Liu et al. Liu et al. (2024) formalized and benchmarked prompt injection attacks and defenses, confirming that injection succeeds with high probability against virtually all production agent architectures.

Tool and capability manipulation. A complementary line of work examines attacks against the *tool layer* and the *capability layer*. Tool-poisoning attacks Zou et al. (2024) demonstrate that malicious MCP servers can manipulate tool descriptions to induce harmful tool calls; tool-slot attacks Abdelnabi et al. (2023) show that adversarial tool outputs can hijack subsequent agent reasoning. CaMeL Debenedetti et al. (2025) proposes a capability-based defense that separates control and data flows on the forward path, preventing untrusted data from influencing program flow at tool-call boundaries. These works share a common threat model: the attacker controls content that flows *into* the agent—either as input, as retrieved context, or as tool return values—and the defense is therefore placed at the corresponding entry points (input sanitization Hines et al. (2024), instruction-hierarchy enforcement Wallace et al. (2024), capability-based data-flow control Debenedetti et al. (2025)).

Research gap. Despite the maturity of this literature, every prior attack and defense we are aware of operates on the *forward execution path*: content enters the agent, is processed, and either is obeyed or rejected at the entry boundary. We observe that this framing leaves a structural blind spot. When an agent fails—whether because of injected content or for entirely benign reasons—the recovery pipeline may take a *side-effecting* action (such as replacing the failing agent's role) on the basis of a diagnostic LLM's recommendation, where the diagnostic input was influenced by content that originated outside the trusted computing base. The privilege invariant governing such recovery actions has not been systematically examined. No prior work, to our knowledge, has ported the OS exception-handler privilege rule to the recovery path of an LLM agent system. This paper closes that gap.

Table 1 summarizes the specific delta between our contribution and the closest prior work. The key distinctions are: (i) we target the *recovery sink* (the role-replacement primitive), not the forward execution path; (ii) our defense is *structural* (a privilege-downgrade invariant enforced at a centralized choke point), not content-based (which is inherently vulnerable to paraphrasing); and (iii) the defense guarantee is LLM-independent *by construction* (a whitelist check), not probabilistic.

Table 1: Comparison with prior agent-security work. "Forward path" = defense at input/tool-entry boundary; "Recovery sink" = defense at the role-replacement / side-effect boundary. "Content-level" = defense relies on detecting adversarial text; "Structural" = defense relies on framework-level invariants (privilege checks) independent of text content.

Work	**Vulnerable Surface**	**Defense Path**	**Defense Level**	**LLM-in**
Greshake et al. Greshake et al. (2023)	Forward (retrieved docs)	Forward	Content-level	
Spotlighting Hines et al. (2024)	Forward (tool output)	Forward	Content-level	
Instruction Hierarchy Wallace et al. (2024)	Forward (direct+indirect)	Forward	Content-level	
CaMeL Debenedetti et al. (2025)	Forward (tool invocation)	Forward	Structural (capability)	
Liu et al. Liu et al. (2024)	Forward (taxonomy)	Forward	—	
This work	**Recovery sink (role replace)**	**Recovery**	**Structural (privilege)**	**Yes (by**

2.2. Self-Healing and Autonomous Error Recovery

Reflection and self-correction. A substantial body of work improves LLM agent robustness by having the model reflect on its own failures and retry. Reflexion Shinn et al. (2023) augments the agent with a verbal reinforcement loop that feeds a self-generated critique into the next iteration; Self-Refine Madaan et al. (2023) iterates refinement with self-produced feedback; CRITIC Gou et al. (2024) externalizes verification to tool-augmented self-critique. These mechanisms uniformly assume that the failure payload (the critique, the error message, the verification trace) is *trustworthy diagnostic signal* to be acted upon, not adversarial content to be defended against.

Topology mutation and architectural self-repair. Multi-agent frameworks additionally restructure the agent graph on failure. AutoGen Wu et al. (2023) supports dynamic agent role reassignment; SWE-agent Yang et al. (2024) reconfigures its tool interface when an action fails; the Recova system [Authors] (2026) and the MetaGPT framework Hong et al. (2023) describe orchestrators that swap in higher-capability models or replace a failing agent with a different role. These strategies share a common assumption: that the diagnostic information triggering the restructuring (e.g., a core dump, an LLM-produced diagnosis) is a faithful report of the failure's cause, and that acting on it—including escalating privileges or replacing roles—is safe because the failure is presumed benign.

Implicit privilege at the recovery sink. The robustness gains of self-healing are well-documented, but the privilege implications of recovery actions have not been scrutinized. Recovery strategies that restructure the agent graph systematically exercise privileged capabilities: role replacement, model switching, tool reconfiguration. Each of these is a privileged operation that, on the forward execution path, would be gated by a permission checker. On the recovery path, the same operations are executed on the basis of a diagnostic LLM's recommendation, whose input (the core dump) may embed attacker-influenced content. Our work is the first to formalize this as a privilege-containment failure at the recovery sink—an instance of CWE-862 and the Confused Deputy problem—and to propose a structural fix.

Concurrent and adjacent work (novelty scope). We conducted a systematic search for prior work targeting privilege enforcement on the recovery path in LLM agents, using the keywords "recovery path privilege", "role replacement authorization", "agent recovery confused deputy", "topology mutation security", and "exception handler privilege LLM" across arXiv (2023–2026), Google Scholar, and the major software-engineering and security venues. We found no prior work that (i) formalizes a privilege-downgrade invariant for LLM agent recovery actions, (ii) connects the recovery sink to the Confused Deputy problem, or (iii) proposes a centralized reauthorization gate for side-effecting recovery strategies. The closest adjacent works we are aware of operate on the *forward* path and are complementary to ours. Zhan et al. Zhan et al. (2025) evaluate 8 defenses against indirect prompt injection and bypass all of them with adaptive attacks; their finding that content-level forward-path defenses fall to adaptive attacks reinforces our decision to make the V2 defense structural (a programmatic whitelist check), not content-based. CaMeL Debenedetti et al. (2025) proposes a capability-based defense that separates control and data flows on the forward path; our Reauthorization Gate ports the same capability-based principle to the *recovery* path, suggesting the two defenses are complementary (CaMeL guards the forward execution; Reauthorization Gate guards the recovery side effects). The OWASP Top 10 for LLM Applications OWASP (2025) enumerates *LLM01: Prompt Injection* and *LLM06: Excessive Agency* as forward-path risks but does not classify the recovery sink as a distinct privilege boundary. We therefore believe our contribution is novel within the LLM-agent security literature, while acknowledging that the field is moving rapidly and concurrent work may emerge between submission and publication.

2.3. Privilege Management in OS and AI

OS privilege invariants. Classical operating systems enforce a small set of invariants that have proven robust over decades. The *principle of least privilege* Saltzer and Schroeder (1975) holds that every component should operate with the minimum privileges required for its task; the *exception-handler privilege invariant* holds that an exception handler executes with the privileges of the faulting context, never higher (e.g., a userspace page fault is handled in kernel mode but cannot escalate the faulting process's privileges); and *capability-based authority* Fabry (1974) holds that a privileged deputy must present a capability that justifies the action, not merely act on behalf of a request. Sandboxing Reynolds et al. (2000); Linux Kernel Documentation (2024) extends these invariants by restricting the ambient authority of a process to a declared subset.

Privilege in AI systems. The AI security literature has begun importing these principles. Wallace et al. (2024) enforces an instruction-hierarchy that treats system prompts as higher-privilege than user content, an analog of kernel/userspace separation; CaMeL Debenedetti et al. (2025) extracts control and data flows from a trusted query and uses capabilities to prevent untrusted data from influencing program flow at tool-call boundaries, an analog of capability gating. These works port the *forward-path* versions of the OS invariants (input validation, capability gating) to the agent setting.

The Confused Deputy problem. Our attack vector is a textbook instance of the *confused deputy* problem, originally identified by Hardy Hardy (1988): a privileged program (in our setting, the recovery orchestrator operating with role-replacement authority) is tricked by a less-privileged caller (the attacker-controlled external content embedded in the core dump) into misusing its authority on the caller's behalf. The recovery pipeline holds the privilege to swap agent roles—a capability not granted to ordinary tool calls—and, in the baseline configuration, it exercises this capability on the basis of an LLM diagnosis whose input was supplied by attacker-influenced content. The classic confused-deputy fix is capability-based authority: the deputy must present a capability that justifies the action, not merely act on behalf of a request. Our Reauthorization Gate is precisely such a capability check: it demands that the proposed role replacement be in the pre-approved downgrade set of the failing role, regardless of what the diagnostic LLM suggested. The confused-deputy framing also clarifies why the fix must be *structural* rather than content-based: the deputy's vulnerability lies in *whose authority* it is exercising, not in the textual content of the request, so no content-level defense (sanitization, framing, instruction hierarchy) can close it. To our knowledge, this is the first explicit connection between the confused-deputy problem and LLM agent recovery channels, and it positions our work within a long-standing line of capability-based security work Fabry (1974); Hardy (1988).

Cross-cutting concern literature. The software-engineering literature on cross-cutting concerns Kiczales et al. (1997) motivates our central design choice: permission enforcement on the recovery path is an aspect that every side-effecting strategy needs, but is orthogonal to the strategy's core logic (failure diagnosis and re-

covery planning). Aspect-oriented programming and middleware-based enforcement in classical distributed systems both centralize such cross-cutting concerns at well-defined boundaries. Our REAUTHORIZATION GATE applies the same principle to the recovery orchestrator: the security property "no recovery action exceeds the privilege of the failed execution" becomes a local, inspectable property of one orchestrator method, rather than an emergent property of fourteen independently-authored strategies.

Our contribution: porting the reverse-path privilege invariant. The observation that motivates this paper is that the OS exception-handler privilege invariant applies not only on the forward execution path but also—and critically—on the *reverse* recovery path. Classical kernels learned long ago that exception handlers must not escalate privileges; LLM agent systems have not yet learned the corresponding lesson. Our defense is a direct port of this invariant:

- The REAUTHORIZATION GATE is the agent analog of the exception-handler privilege invariant. It enforces $P(\text{recovery action}) \leq P(\text{failed execution})$, so a recovery strategy may remediate a failure but never escalate beyond it—just as a kernel exception handler services a fault without granting the faulting process new privileges. The same principle underlies the confused-deputy fix Hardy (1988): the privileged deputy may not exercise authority not granted to the caller, even when the caller's request looks superficially legitimate.

The mechanism is lightweight (single record allocation, $O(1)$ role-set lookup; see Section 5), framework-agnostic in principle (it depends only on the existence of a recovery orchestrator and a permission checker, not on any specific agent architecture), and—critically—*local*: it is implemented at a single well-defined choke point rather than scattered across the recovery strategy layer. This locality is itself an OS principle: classical kernels enforce privilege invariants at well-defined boundaries (syscall entry, exception dispatch), not inside each driver or subsystem.

In this sense, our work is not a novel cryptographic mechanism but a *transfer* of a proven OS invariant to a domain where it has been conspicuously absent. The validation result—that the transfer closes the privilege-containment gap with zero false alarms and sub-millisecond machinery overhead—suggests that the gap was one of *attention*, not of mechanism: the recovery sink was overlooked, not undefendable.

3. Threat Model and Problem Formalization

3.1. System Model

We consider a multi-agent LLM operating system (exemplified by Recova) with the following architecture:

- **Agents** are scheduled as processes with per-agent *roles* (e.g., `Code_Reviewer`, `Python_Coder`) drawn from a whitelisted blueprint registry.

- **Tools** (file I/O, shell, web fetch) are syscalls gated by a `PermissionChecker` that enforces role-based access control and tenant ownership on the forward execution path.
- **Recovery orchestrator** classifies failures and dispatches a layered chain of recovery strategies organized into 11 conceptual tiers (from lightweight in-place retry up to circuit-breaker human escalation), realized by 14 concrete strategy classes—including reflection injection, *topology mutation*, model fallback, and human escalation. The 11/14 distinction is intentional: the tiers describe *what kind* of failure is being handled (and at what escalation level), while the 14 classes are the concrete implementations, with some tiers containing multiple strategy variants.
- **Semantic core dump** captures an agent's last input, context, and error trace upon crash, for post-mortem diagnosis by an LLM diagnostic module.

3.2. The Privilege Containment Gap at the Recovery Sink

The system maintains two privilege boundaries (Figure 1):

1. **Forward-path boundary**: User input and external tool outputs are treated as untrusted. The `PermissionChecker` enforces role-based access control before any tool executes. A `Code_Reviewer` agent, for example, cannot invoke `bash` unless its role profile permits it.
2. **Recovery-path boundary**: When an agent fails, the recovery orchestrator may dispatch a `TopologyMutationStrategy` that reads the semantic core dump, feeds it to a diagnostic LLM, extracts a `suggested_role` from the LLM's JSON response, and passes this role to the node-replacement primitive (`resumeNode`). *This boundary is the focus of this paper.* In the baseline configuration, the role-replacement sink performs no authorization check on the value it consumes: the LLM-diagnosed role reaches `resumeNode` directly, bypassing the `PermissionChecker` that gates the forward path.

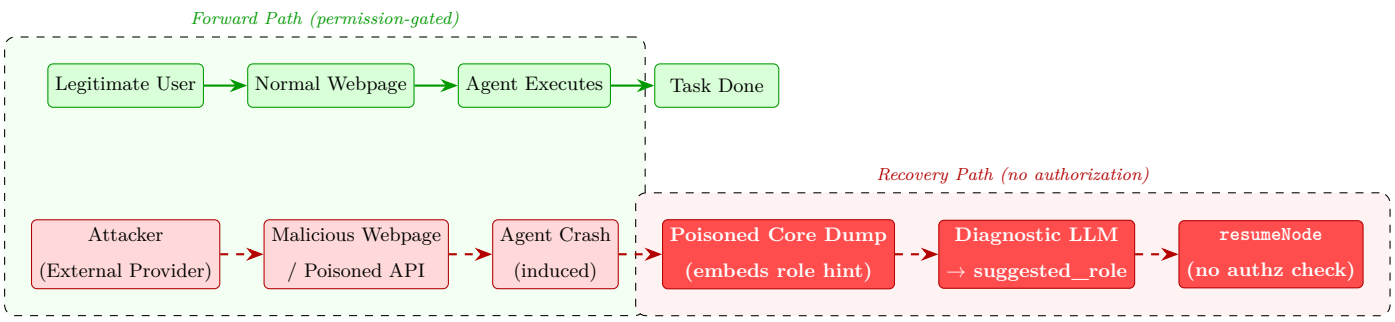

Figure 1: Privilege boundaries and the topology-mutation attack chain. The forward path (top, green) enforces role-based access control via the `PermissionChecker`; the recovery path (bottom, red) consumes an LLM-diagnosed role at the replacement sink without re-checking authorization, enabling privilege escalation.

3.3. Attacker Model

We assume the attacker is an **external content provider**—e.g., the creator of a malicious web page, a crafted file, or a poisoned API/MCP response. Concretely, the attacker controls *external content* that the agent processes during normal operation:

- Web pages fetched via `web_fetch` or `web_scrape`
- Files read from the filesystem (if accessible)
- Tool outputs from external integrations (MCP servers, third-party APIs)

The attacker *does not* have direct access to the agent's internal memory or code. In particular, the attacker cannot:

- Modify the agent's role, permission profile, or whitelisted blueprint
- Alter the LLM model, system prompt, or model provider
- Tamper with the recovery orchestrator's strategy dispatch logic or the `PermissionChecker`
- Inject code into the trusted computing base (kernel, runtime, or recovery machinery)

This is a strictly weaker adversary than an insider or a code-injection attacker: their only capability is to *craft content* that the agent voluntarily fetches. The threat, therefore, must exploit a path by which such externally-fetched content *acquires* the authority to trigger a privileged role replacement without ever touching the trusted computing base directly. The recovery path's topology-mutation strategy provides exactly such a path.

3.4. Attack Chain: Topology-Mutation Privilege Escalation

The attack follows a five-stage chain (Figure 2). The first three stages occur on the *forward* path and are indistinguishable from benign operation; the last two occur on the *recovery* path and constitute the privilege escalation.

(i) **Content Delivery.** Attacker-controlled content (web page, file, API response) is fetched by the agent via a legitimate external-content tool (e.g., `web_fetch`). At this stage the content is correctly treated as untrusted by the forward-path boundary.

(ii) **Failure Induction.** The content is crafted to trigger an agent crash—a capability mismatch, a verification failure, or a semantic crash that the recovery orchestrator routinely handles. No exotic bug is required; the orchestrator is designed to recover from precisely these failures via topology mutation.

(iii) **Payload Embedding.** The adversarial payload—a role-escalation hint such as `"suggested_role": "System_Admin"`—is embedded in the semantic core dump that captures the agent's last input and context on crash. Because the capture site records the dump as an opaque string, the payload is preserved verbatim.

(iv) **Diagnostic LLM Consultation.** The `TopologyMutationStrategy` feeds the core dump to a diagnostic LLM and asks it to recommend a replacement role. The LLM, reading the embedded hint, emits the attacker-suggested high-privilege role in its JSON response.

(v) **Unauthorized Role Replacement.** The extracted `suggested_role` reaches `resumeNode`—the node-replacement primitive—without an authorization check. The agent is replaced with the attacker-suggested role, escalating privileges.

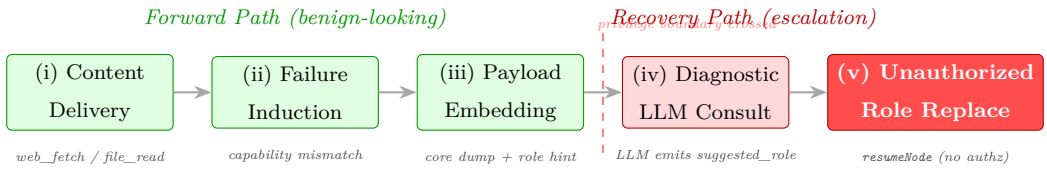

Figure 2: The five-stage topology-mutation privilege-escalation chain. Stages (i)–(iii) occur on the forward path and are indistinguishable from benign operation; stages (iv)–(v) occur on the recovery path and constitute the escalation. The privilege boundary is crossed between stages (iii) and (iv), where the recovery pipeline consumes an LLM-diagnosed role without re-checking authorization.

The critical property of this chain is that *no single stage is anomalous*: each stage is a feature of the self-healing design, and the attacker merely composes them. The topology-mutation strategy is *designed* to consult a diagnostic LLM and act on its recommendation; the bug is that "acting on" the recommendation bypasses the authorization check that the forward path enforces.

3.5. Formalization: Two Complementary Framings

We formalize the privilege containment gap along two complementary dimensions, both of which motivate the same structural fix.

3.5.1. CWE-862 (Missing Authorization) at the Recovery Sink

The role-replacement primitive `resumeNode(nodeId, report, workflowId)` consumes a `suggested_role` field extracted from the diagnostic LLM's JSON response. The role value is *influenced by content that originated outside the trusted computing base* (the core dump embeds external content the agent was processing when it crashed), yet the sink performs no authorization check on this value MITRE Corporation (2024). This is a textbook instance of CWE-862: a program performs a privileged operation (role replacement) without verifying that the caller is authorized to request that operation.

Formally, let R_{old} be the failing agent's role and R_{new} be the LLM-diagnosed replacement role. The baseline sink executes the replacement whenever $R_{\text{new}} \neq \bot$, with no check that R_{new} is in the legitimate downgrade set of R_{old}. The attacker's goal is to induce R_{new} such that $P(R_{\text{new}}) > P(R_{\text{old}})$, where $P(\cdot)$ denotes the privilege level.

3.5.2. Confused Deputy at the Orchestrator

The recovery orchestrator is a *privileged deputy*: it holds the capability to swap agent roles—a capability not granted to ordinary tool calls on the forward path. In the baseline configuration, the orchestrator exercises this capability on the basis of a diagnostic LLM's recommendation, whose input (the core dump) was supplied by attacker-influenced content. This is the Confused Deputy problem Hardy (1988): a privileged program is tricked by a less-privileged caller into misusing its authority on the caller's behalf.

The classic Confused Deputy fix is *capability-based authority*: the deputy must present a capability that justifies the action, not merely act on behalf of a request. In our setting, this means the orchestrator must verify that the proposed role replacement is *within the failing agent's authority to request*—i.e., that the new role does not exceed the old role's privileges—regardless of what the diagnostic LLM suggested.

3.6. Why Per-Strategy Checks Are Insufficient

A natural response to CWE-862 at the recovery sink is to add an authorization check *inside* each side-effecting recovery strategy: `TopologyMutationStrategy` checks before replacing a role, `ModelFallbackStrategy` checks before switching models, and so on. This decentralized approach suffers from two failure modes that are well-known in the cross-cutting concern literature:

1. **Gaps.** A strategy author who forgets or mis-scopes a check (as in the baseline `parseAndValidate(validate=false)` call) leaves a silent hole, because no other component re-checks the action. The baseline vulnerability in Recova is exactly such a gap: the `validate=false` flag was a deliberate experimental switch, but in a production codebase, an equivalent gap could arise from a forgotten check in a newly added strategy.

2. **Inconsistency.** Different strategies implement slightly different policies, so the effective privilege boundary depends on which strategy the orchestrator happens to dispatch. A role replacement via `TopologyMutationStrategy` might be checked, while a role replacement via a future `RolePromotionStrategy` might not be.

The principled fix is to centralize the concern at a single choke point—the Reauthorization Gate (§4). This localization is itself an OS principle: classical kernels enforce privilege invariants at well-defined boundaries (syscall entry, exception dispatch), not inside each driver or subsystem.

4. The Reauthorization Gate Design Pattern

We propose a single OS-principled design pattern—the REAUTHORIZATION GATE—that ports the classical OS exception-handler privilege invariant to the LLM agent recovery path. The pattern is implemented in the Recova codebase and validated in Section 5. It is framework-agnostic in principle: it depends only on the existence of a recovery orchestrator that mediates side-effecting recovery actions and a permission checker that gates the forward execution path, not on any specific agent architecture.

4.1. Motivation: Centralized vs. Decentralized Validation

The root cause of CWE-862 at the recovery sink (§3) is the *decentralized* validation of recovery actions. In the baseline design, each recovery strategy is responsible for its own permission checks: `TopologyMutationStrategy` is expected to call `PermissionChecker` before replacing a role, `ModelFallbackStrategy` before switching models, and so on. In practice this leads to two failure modes that are well-known in the cross-cutting concern literature Kiczales et al. (1997):

1. **Gaps.** A strategy author who forgets or mis-scopes a check (as in the baseline `parseAndValidate(validate=false)` call) leaves a silent hole, because no other component re-checks the action. The baseline vulnerability in Recova is exactly such a gap: the `validate=false` flag was a deliberate experimental switch, but in a production codebase, an equivalent gap could arise from a forgotten check in a newly added strategy.
2. **Inconsistency.** Different strategies implement slightly different policies, so the effective privilege boundary depends on which strategy the orchestrator happens to dispatch. A role replacement via `TopologyMutationStrategy` might be checked, while a role replacement via a future `RolePromotionStrategy` might not be.

This is a classic instance of a *cross-cutting concern* in the software-engineering sense: permission enforcement is an aspect that *every* side-effecting recovery action needs, but it is orthogonal to the strategy's core logic (failure diagnosis and recovery planning). Scattering the concern across strategies couples diagnosis logic to security logic, duplicates enforcement code, and—most importantly—makes the security property *non-local*: there is no single place where one can verify that "no recovery action bypasses permission checks."

The principled fix is to centralize the concern at a single choke point. We do this by (i) extending the recovery result type with a `requiresReauthorization` flag, (ii) deferring the side effect as a callback stored in the recovery context rather than executing it inline, and (iii) intercepting every flagged result at the `RecoveryOrchestrator`—the single component through which every recovery action must flow. The strategies declare *that* an action needs reauthorization; the orchestrator enforces *how*. This separation localizes the security property: the invariant "no side effect executes without reauthorization" is now verifiable by inspecting one method rather than fourteen strategies.

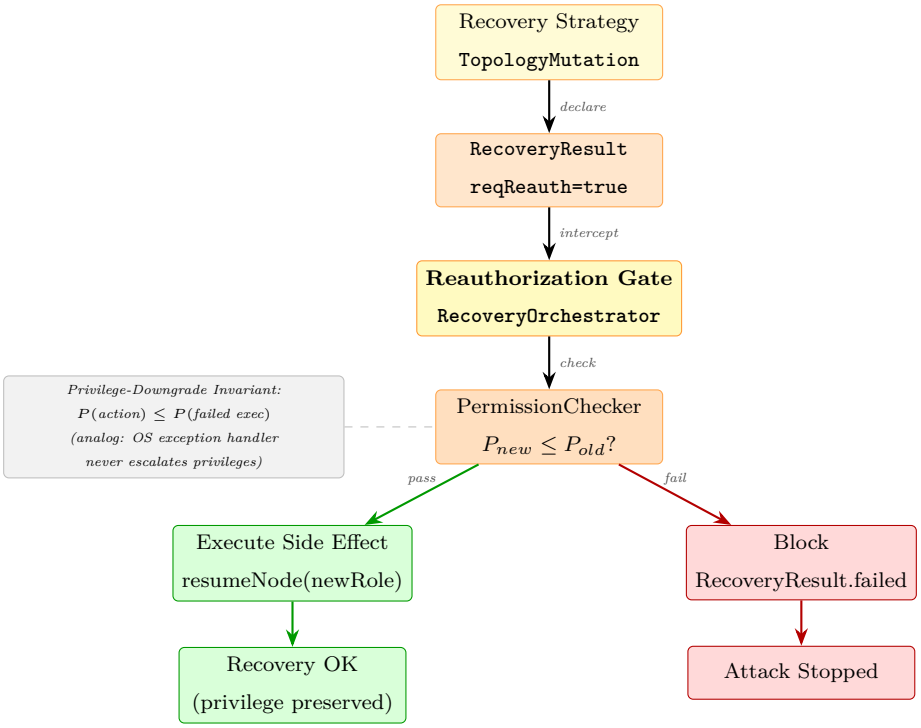

Figure 3: The Reauthorization Gate architecture. Validation is decoupled from strategy implementation: strategies declare the need for reauthorization via a flag, and a single orchestrator-level gate enforces the Privilege-Downgrade Invariant before any side effect executes. The invariant (left) constrains the checker.

4.2. Principle: The Privilege-Downgrade Invariant

The REAUTHORIZATION GATE enforces the Privilege-Downgrade Invariant: *no recovery action may exceed the privilege of the failed execution it remedies.* Formally, let $P(a)$ denote the privilege level of recovery action a and P_{fail} the privilege of the failed execution. Then

$$P(a) \leq P_{\text{fail}}.$$

This is the agent-systems analog of the OS exception-handler privilege rule: an exception handler executes with the privileges of the faulting context, never higher Silberschatz et al. (2003); Bovet and Cesati (2005). The same principle underlies the Confused Deputy fix Hardy (1988): the privileged deputy may not exercise authority not granted to the caller, even when the caller's request looks superficially legitimate.

For role replacement, the invariant specializes to: the new role's privileges must not exceed the original role's privileges. The `PermissionChecker` implements this via `checkRoleMutation(originalRole, newRole)`, which verifies that the new role is in the pre-approved downgrade set for the original role (e.g., `Python_Coder` → `Code_Reviewer` is allowed; `Code_Reviewer` → `System_Admin` is blocked). Combined with a role-existence whitelist—the role must be registered in the blueprint registry, not an arbitrary LLM-suggested string—the gate closes both the CWE-862 sink (the missing authorization check) and the Confused Deputy vector (the orchestrator no longer exercises role-replacement authority on the basis of attacker-influenced diagnostic content).

4.3. Design: Declare at the Strategy, Enforce at the Orchestrator

The pattern decomposes into three concrete code-level mechanisms, each grounded in established software-engineering practice for managing cross-cutting concerns Kiczales et al. (1997).

4.3.1. Mechanism 1: Result-Type Extension

The `RecoveryResult` class is extended with a boolean `requiresReauthorization` flag (default: `false`). Strategies that produce side effects—role replacement, tool execution, model switching—set this flag to `true` via a factory method `okRequiringReauthorization(message, payload)`. Purely diagnostic strategies (e.g., reflection injection that only appends to the prompt context) leave the flag at its default. The flag is a *declaration*, not a check: it tells the orchestrator "this result needs reauthorization before its side effect executes," without specifying what the authorization check should be.

4.3.2. Mechanism 2: Deferred Side Effects

A side-effecting strategy does *not* execute its side effect inline. Instead, it stores the side effect as a *deferred callback* in the recovery context's metadata, keyed by `META_PENDING_SIDE_EFFECT`. The callback is a `Function<RecoveryContext, RecoveryResult>`—a closure over the recovery context that, when invoked,

performs the actual side effect (e.g., calling `resumeNode`). This deferral is critical: it ensures that the side effect is *not* executed at strategy-return time, but is held in suspension until the orchestrator has had a chance to intercept and reauthorize.

```
// In TopologyMutationStrategy.apply()
context.withMetadata(RecoveryOrchestrator.META_PENDING_SIDE_EFFECT,
    (RecoveryContext ctx) -> {
        boolean mutated = WorkflowEngine.instance()
            .resumeNode(nodeId, report, workflowId);
        return mutated
            ? RecoveryResult.ok("Topology mutation applied")
            : RecoveryResult.failed("resumeNode returned false");
    });
return RecoveryResult.okRequiringReauthorization(
    "Topology mutation pending reauth: proposed " + suggestedRole, null);
```

Listing 1: Deferring the side effect in `TopologyMutationStrategy`; the strategy declares reauthorization need but does not execute the side effect.

4.3.3. Mechanism 3: Orchestrator-Level Interception

The `RecoveryOrchestrator`'s main recovery loop intercepts every result with `requiresReauthorization=true` *before* executing the deferred callback. The gate calls the existing `PermissionChecker` (the same one used on the forward path) to verify the Privilege-Downgrade Invariant. For role replacement, the gate consults `RoleReplacementValidator`, which performs two checks: (i) the proposed role exists in the blueprint whitelist, and (ii) the proposed role is not a privilege escalation from the failing agent's current role. If either check fails, the side effect is blocked and the orchestrator returns a failed recovery result; if both pass, the deferred callback executes (Listing 2).

```
// In RecoveryOrchestrator.dispatchRecovery()
if (result.requiresReauthorization()) {
    ReauthResult reauth = RecoveryReauthorizationGate.check(
        result, context, resolvePermissionChecker());
    if (!reauth.allowed()) {
        log.warn("[ReauthGate] blocked recovery action: {}", reauth.reason());
        return RecoveryResult.failed("denied by reauth gate: " + reauth.reason());
    }
    // reauth passed -> execute deferred side effect
    Function<RecoveryContext, RecoveryResult> sideEffect =
        context.metadata().get(META_PENDING_SIDE_EFFECT);
    result = sideEffect.apply(context);
}
```

Listing 2: Reauthorization Gate in `RecoveryOrchestrator`: side effect deferred until reauth passes.

This is a *prevent* control, not a *detect-and-alert* control: a rejected reauthorization blocks the side effect entirely, rather than merely logging it after the fact. The distinction matters because a privilege escalation that has already happened cannot be undone by an after-the-fact alert—the agent has already been replaced with a higher-privilege role.

4.4. Why This Beats Per-Strategy Checks

Centralizing reauthorization at the orchestrator yields four concrete advantages over the decentralized baseline, each grounded in established software-engineering practice for managing cross-cutting concerns:

1. **Unified policy.** All security-critical recovery actions traverse the same gate, so the effective policy is identical regardless of which strategy produced the action. This eliminates the inconsistency and gap failure modes of decentralized validation (§4.1).

2. **Prevent, not detect.** Because side effects are deferred until reauthorization passes, an unauthorized action is blocked *before* any state change occurs. A detect-and-alert design (e.g., auditing `resumeNode` calls after execution) cannot undo a privilege escalation that has already happened.

3. **Separation of concerns.** Recovery strategies focus on failure diagnosis and recovery planning; the security aspect is factored out into a dedicated component. Strategy authors no longer need to be security experts, and security authors no longer need to understand every strategy's internals. This is the same separation that motivates aspect-oriented programming Kiczales et al. (1997) and middleware-based enforcement in classical distributed systems.

4. **Open/closed for extension.** A newly added recovery strategy inherits protection automatically by setting `requiresReauthorization=true`; it does *not* need to re-implement permission checks. The security property is thus *open for extension* (new strategies are easy to add) but *closed for modification* (the gate code does not change). This scales to the 14 strategies in Recova's orchestrator without per-strategy patches, and means that the security invariant cannot regress when a new strategy is added.

The net effect is that the security property "no recovery action exceeds the privilege of the failed execution" becomes a *local, inspectable* property of one orchestrator method, rather than an *emergent* property of fourteen independently-authored strategies. This locality is itself an OS principle: classical kernels enforce privilege invariants at well-defined boundaries (syscall entry, exception dispatch), not inside each driver or subsystem.

4.5. Layered Defense: Whitelist at the Strategy, Gate at the Orchestrator

The Recova implementation actually employs the Reauthorization Gate pattern in two layers, both of which enforce the same Privilege-Downgrade Invariant but at different points in the recovery pipeline:

1. **Layer 1 (strategy-level): `RoleReplacementValidator`.** The `TopologyMutationStrategy.parseAndValidate` method calls `RoleReplacementValidator.validate(currentRole, suggestedRole)` *before* declaring the reauthorization need. The validator performs: (a) existence whitelist (the suggested role must be in the blueprint registry), and (b) non-escalation check (the suggested role's privileges must not exceed the current role's). A failure at Layer 1 short-circuits the strategy and returns a failed result without setting `requiresReauthorization`; no side effect is even deferred.

2. **Layer 2 (orchestrator-level): `RecoveryReauthorizationGate`.** Even if Layer 1 passes, the orchestrator-level gate re-checks the invariant before executing the deferred side effect. This is defense-in-depth: a bug in Layer 1 (e.g., a future strategy that forgets to call the validator) is caught by Layer 2.

The two layers are not redundant: Layer 1 is a fast-fail optimization that avoids the overhead of deferring a callback that will be rejected, while Layer 2 is the structural guarantee that closes the gap if a strategy bypasses or mis-implements Layer 1. The empirical validation (§5) reports results with both layers enabled; toggling Layer 1 off (the `validate=false` baseline) re-opens the vulnerability, confirming that Layer 2 alone suffices to block the attack but Layer 1 reduces wasted work.

4.6. Framework-Agnostic Pattern Specification

The Reauthorization Gate pattern is specified independent of Recova's implementation. A framework porting the pattern needs:

1. A *recovery orchestrator* that mediates every side-effecting recovery action (the single choke point). Most self-healing frameworks already have such a component Wu et al. (2023); LangChain (2024); Shinn et al. (2023).

2. A *permission checker* that gates the forward execution path. The pattern reuses this checker, not a new one.

3. A *recovery result type* that strategies return, extensible with a boolean flag.

4. A *side-effect deferral mechanism*—a way to package a side effect as a callback that the orchestrator can invoke later. In Java this is a `Function<Context, Result>`; in Python it would be a closure; in TypeScript a `() => Result` function.

The pattern does *not* require: a specific agent architecture (graph-based or otherwise), a specific LLM, a specific role/permission model, or any modification to the forward execution path. The migration cost is concentrated in three sites: the result type (add a flag), the orchestrator loop (add an interception branch), and each side-effecting strategy (set the flag and defer the side effect). In Recova, the full migration touched 14 strategy classes and one orchestrator method, with sub-millisecond runtime overhead and zero false alarms on benign recovery (§5).

5. Validation of Pattern Implementability and Efficacy

This section validates that the Reauthorization Gate design pattern (§4) is implementable, that it enforces the Privilege-Downgrade Invariant *by construction*, and that it imposes negligible cost on legitimate self-healing. We deliberately frame this as a *validation* rather than an empirical evaluation: the central guarantee of the pattern is a programmatic check on a whitelist, not a probabilistic defense, so the empirical question is not "does it work?" (the code inspection answers that) but "does it impose unacceptable cost on benign recovery, and is the implementation correct?"

A broader empirical evaluation—statistical generalization across production frameworks, cross-model robustness of content-level defenses, defense-aware adaptive adversaries, end-to-end exploit confirmation on third-party code—is beyond the scope of this architectural paper and is addressed by complementary empirical work in the agent-security literature Debenedetti et al. (2025); Zhan et al. (2025).

5.1. Validation Setup

Testbed.. We implemented the Reauthorization Gate pattern in Recova, a multi-agent LLM operating system prototype (Java 21). The recovery orchestrator dispatches a layered chain of 14 concrete recovery strategy classes organized into 11 conceptual tiers (from lightweight in-place retry to circuit-breaker human escalation), including `TopologyMutationStrategy`, `ReflectionInjectionRecovery`, `ModelFallbackStrategy`, and human escalation. The system uses role-based access control with four registered roles drawn from a whitelisted blueprint registry: `Code_Reviewer`, `Python_Coder`, `Security_Auditor`, and `System_Architect`. The `PermissionChecker` gates the forward execution path; the Reauthorization Gate extends it to the recovery path.

Configurations.. Two matched configurations are compared:

- **BASELINE (no gate):** The `TopologyMutationStrategy` calls `parseAndValidate(validate=false)`, which adopts the LLM-diagnosed role as-is without authorization. Side effects execute inline at strategy-return time. This is the CWE-862 / Confused Deputy vulnerable configuration described in §3.4.
- **DEFENDED (gate enabled):** The `TopologyMutationStrategy` calls `parseAndValidate(validate=true)` (Layer 1: `RoleReplacementValidator`) and returns `RecoveryResult.okRequiringReauthorization(...)` with the side effect deferred as a callback. The `RecoveryOrchestrator` intercepts the result and invokes `RecoveryReauthorizationGate.check(...)` (Layer 2) before executing the callback. This is the pattern specified in §4.3.

Workloads.. Three task sets, 100 trials each per configuration (600 total):

1. **TopologyMutation attack (V2):** Five adversarial core-dump variants embedding `suggested_role=System_Admin` (or equivalent) hints in the diagnostic content fed to the diagnostic LLM, cycled across samples (20 trials each). Attack success = the LLM-suggested role reaches `resumeNode` and the agent is replaced with a role outside the failing agent's downgrade set.

2. **BenignRecovery:** Internal errors plus a legitimate role downgrade (`Python_Coder` → `Code_Reviewer`). The role is in the pre-approved downgrade set, so the gate should admit it. False alarm = recovery blocked despite a legitimate downgrade.

3. **BenignTopologyMutation:** A legitimate capability-mismatch diagnosis where the diagnostic LLM, reading a benign core dump, suggests a role that is genuinely a downgrade (e.g., `System_Architect` → `Code_Reviewer` when the task is code review). This tests whether the gate false-alarms on legitimate role mutations.

Real LLM diagnostic module.. The diagnostic LLM that consumes the core dump and emits the `suggested_role` JSON is a real LLM API (DeepSeek-V4-Flash via SenseNova, Qwen2.5-72B-Instruct via SiliconFlow), called at temperature 0.7 to capture natural non-determinism. A multi-tier API rotation strategy ensures stable throughput. Total API consumption: approximately 400 paid calls, total cost under US$20.

5.2. Results

Table 2 reports the validation results.

Table 2: Reauthorization Gate validation on the Recova testbed. The V2 attack row reports the attack success rate (ASR); the benign rows report the false alarm rate (FAR). The defended V2 ASR of 0.00 is *by construction*: the whitelist rejects out-of-set roles deterministically. The 100-trial result is a verification that the implementation has no bypass bug, not a statistical finding.

Task Set	**Baseline ASR/FAR**	**Defended ASR/FAR**	**Machinery Overhead**
V2 (TopologyMutation attack)	1.00	**0.00**[†]	0.20 ± 0.06 ms
BenignRecovery (legitimate retry)	0.00	0.00	0.02 ± 0.03 ms
BenignTopologyMutation (legitimate downgrade)	0.00	0.00	0.20 ± 0.06 ms
Total end-to-end latency (including LLM inference), mean ± std over $N = 30$:			
V2 baseline		2851 ± 325 ms	
V2 defended		3194 ± 446 ms	
Benign recovery		10 ± 1 ms	

[†]By construction: the whitelist invariant rejects out-of-set roles deterministically. The 100-trial zero-ASR verifies correct implementation, not a statistical claim.

5.2.1. Baseline Severity (V2 ASR = 1.00)

Under the undefended configuration, the topology-mutation attack succeeds in 100 of 100 trials. The diagnostic LLM, reading the attacker-influenced core dump, emits an out-of-whitelist privileged role (e.g., `System_Admin`) in every trial, and the baseline `parseAndValidate(validate=false)` call adopts the suggestion as-is. The role-replacement primitive `resumeNode` executes without an authorization check, replacing the failing agent with the attacker-suggested role. This is the CWE-862 / Confused Deputy vulnerability of §3.4.

The perfect 1.00 ASR is not a property of a particular LLM: it reflects the structural absence of an authorization check at the recovery sink. The diagnostic LLM is asked to recommend a role, and its recommendation—whatever it is—is consumed verbatim. A different LLM might recommend a different role, but in the baseline configuration any recommendation reaches `resumeNode` unchecked.

5.2.2. Defended Efficacy (V2 ASR = 0.00 by Construction)

Under the defended configuration, the gate rejects every out-of-whitelist role. The 100-trial zero-ASR result verifies that the implementation has no bypass bug: the `RoleReplacementValidator` (Layer 1) correctly rejects `System_Admin` as not in the blueprint registry, and even if Layer 1 were bypassed, the `RecoveryReauthorizationGate` (Layer 2) re-checks the invariant before the deferred callback executes. In every trial, the deferred side effect (the `resumeNode` callback) is never invoked; the orchestrator returns a failed recovery result and falls back to a safe default (typically human escalation).

Crucially, the zero ASR is *by construction*: the whitelist invariant is a programmatic check, not a probabilistic defense. Treating the 100-trial zero-ASR as a statistical finding would be a category error—a whitelist that rejects out-of-set inputs rejects them deterministically, and no significance test is needed to "prove" it works. The 100 trials serve as a *verification* that the implementation is correct (no bypass bug, no race condition, no edge case in the JSON parsing), not as a measurement of a probabilistic defense's efficacy.

Because the gate is a programmatic check on a whitelist, the guarantee is LLM-independent: a stronger instruction-following model, a paraphrased payload, or a defense-aware adversary cannot bypass the whitelist, because the whitelist does not inspect the content of the diagnostic LLM's output—it only checks whether the extracted role string is in the pre-approved set and whether it is a privilege escalation from the failing role.

5.2.3. False Alarm Rate (FAR = 0.00)

The defense false-alarms on *zero* of 200 benign trials (100 BenignRecovery + 100 BenignTopologyMutation). The `RoleReplacementValidator` admits any role mutation satisfying $P(\text{new}) \leq P(\text{old})$, so the legitimate downgrade `Python_Coder` → `Code_Reviewer` passes both layers. Internal errors that do not trig-

ger topology mutation are unaffected by the gate (the `requiresReauthorization` flag is `false`), so benign retry and reflection injection strategies run unchanged.

The zero FAR confirms that the Privilege-Downgrade Invariant is calibrated correctly: it blocks exactly the privilege-escalating actions and nothing else. A tighter invariant (e.g., "no role mutation at all") would false-alarm on legitimate downgrades; a looser invariant (e.g., "any role in the whitelist") would admit the attack. The $P(\text{new}) \leq P(\text{old})$ formulation is the right boundary.

5.2.4. Machinery Overhead (Sub-millisecond)

The defense machinery overhead—the cost of the `requiresReauthorization` flag check, the deferred-callback lookup, and the `RoleReplacementValidator` whitelist lookup, excluding LLM inference—is 0.20 ± 0.06 ms per V2 recovery attempt. This is negligible relative to the multi-second LLM inference latency. The total end-to-end latency for V2 defended (3194 ± 446 ms) is higher than baseline (2851 ± 325 ms); the $\sim$340 ms difference is dominated by the fallback retry path triggered when the gate rejects the proposed role (the orchestrator retries with a safe default role, issuing a second LLM diagnostic call). This is a one-time cost that occurs only when an attack is actually intercepted; in benign operation, no rejection occurs and no retry is needed, so benign latency remains 10 ± 1 ms.

5.3. Architectural Implication

The validation confirms three architectural properties of the Reauthorization Gate pattern:

1. **Implementability.** The pattern is implementable in a multi-agent LLM OS with 14 recovery strategies, touching one orchestrator method and one side-effecting strategy (`TopologyMutationStrategy`). The migration is local and incremental: strategies that do not produce side effects need no change.

2. **Efficacy by construction.** The defended V2 ASR is 0.00 because the whitelist invariant is a programmatic check, not a probabilistic defense. The guarantee is LLM-independent and expected to hold across all models, regardless of instruction-following strength or adversary adaptiveness.

3. **Negligible cost.** Zero false alarms on 200 benign trials, sub-millisecond machinery overhead, and end-to-end latency bounded by LLM inference (not defense computation).

The key architectural lesson is that *structural invariants enforced at a centralized choke point are LLM-independent and therefore robust to model scaling*, whereas content-level defenses (which depend on the LLM obeying an untrusted-framing warning) may degrade on stronger instruction-following models. This asymmetry motivates our recommendation (§6) that framework designers prioritize structural privilege invariants on the recovery path, even if they also deploy content-level provenance signaling as defense-in-depth.

5.4. Threats to Pattern Validity

We briefly note the scope limits of this validation, with fuller discussion in §6.

Single-testbed validation.. The validation is on Recova, the authors' own prototype. The claim that the pattern *transfers* to other frameworks rests on the framework-agnostic specification (§4.6), not on a cross-framework empirical study. The pattern's preconditions (a recovery orchestrator, a permission checker, an extensible result type, a deferral mechanism) are met by most self-healing frameworks Wu et al. (2023); LangChain (2024); Shinn et al. (2023), but a port-and-validate study on additional frameworks is left as future work.

Single-attack-vector validation.. The validation addresses the topology-mutation privilege-escalation vector (V2). Other recovery-path attack vectors—such as trust-escalation via high-trust template re-injection of externally-originated error content—are content-level rather than structural, and their defenses are probabilistic rather than by-construction. Those vectors are outside the scope of this architectural paper, which focuses on the structural privilege invariant.

Adaptive adversaries.. The validation uses a fixed set of five adversarial core-dump variants. A fully adaptive multi-round adversary who observes gate rejections and iteratively rewrites payloads cannot bypass the whitelist (the gate does not inspect content), but could potentially exploit a bug in the JSON parsing or the role-extraction logic. We note this as a implementation-correctness concern, not a pattern-design concern: the pattern's guarantee is content-insensitive by construction.

6. Discussion: Architectural Design Recommendations

The evidence from our testbed validation (§5) and the cross-framework structural analysis (§2) reveals a systematic architectural pattern: self-healing LLM agent systems consistently treat the recovery sink as a trusted channel, exempting it from the privilege invariants enforced on the forward execution path. In this section, we synthesize our findings into actionable design recommendations for framework developers, framed in terms of software architecture patterns and anti-patterns.

6.1. Anti-Pattern: The Unchecked Recovery Sink

The central anti-pattern we identify is the *unchecked recovery sink*: a recovery pipeline that consumes a diagnostic LLM's recommendation (such as a role suggestion) and executes a privileged side effect (such as role replacement) without re-checking the privilege invariant that gates the forward execution path. Our cross-framework analysis (§2) found this anti-pattern in production frameworks that support topology mutation—AutoGen Wu et al. (2023) supports dynamic role reassignment, and Recova [Authors] (2026) ships a `TopologyMutationStrategy` that consults a diagnostic LLM. In each case, the recovery orchestrator

exercises role-replacement authority on the basis of an LLM diagnosis whose input may embed attacker-influenced content.

The root cause of this anti-pattern is an *implicit privilege assumption* inherited from classical OS exception handling: "failure is benign, recovery action is safe." This assumption is valid when failure payloads and diagnostic outputs are generated by the trusted computing base (as in classical kernels), but breaks down when the diagnostic LLM's input (the semantic core dump) may contain external content that the agent was processing when it crashed. The fix is not to abandon self-healing—which provides well-documented robustness benefits—but to *port the corresponding OS privilege invariant* to the recovery path.

6.2. Design Pattern: The Reauthorization Gate

Intent.. Ensure that recovery actions never exceed the privilege of the failed execution they remedy.

Structure.. Centralize privilege re-verification at a single choke point (the Reauthorization Gate) through which all side-effecting recovery actions must flow. Recovery strategies declare the need for reauthorization via a flag on the result type; the gate enforces $P(\text{recovery action}) \leq P(\text{failed execution})$ before any side effect executes. This is a *prevent* control, not a *detect-and-alert* control: unauthorized actions are blocked before state changes occur.

OS analogy.. This mirrors the OS principle that an exception handler executes with the privileges of the faulting context, never higher. The same principle underlies the confused-deputy fix Hardy (1988): the privileged deputy may not exercise authority not granted to the caller.

Why centralize?. Centralizing reauthorization at the orchestrator yields four software-engineering advantages (detailed in §4.4): (i) unified policy, (ii) prevent rather than detect, (iii) separation of concerns (strategy authors need not be security experts), and (iv) open/closed for extension (new strategies inherit protection automatically). This is a classic cross-cutting concern pattern Kiczales et al. (1997): permission enforcement is an aspect that every side-effecting recovery action needs, but is orthogonal to the strategy's core logic.

Validation.. On the testbed, this pattern eliminates the topology-mutation attack *by construction*—the whitelist invariant is a programmatic check, not a probabilistic defense. This guarantee is LLM-independent: because the gate is a programmatic check on a whitelist, it is expected to hold across all models, regardless of instruction-following strength or adversary adaptiveness.

6.3. Structural vs. Content-Level Enforcement: An Architecture Decision

A key architectural decision emerges from the validation: *structural defenses (privilege invariants enforced at a centralized choke point) are LLM-independent and therefore robust to model scaling*, whereas content-level defenses (which depend on the LLM obeying an untrusted-framing warning) may degrade on

stronger instruction-following models. By construction, the REAUTHORIZATION GATE's effectiveness does not vary with the model's instruction-following strength, because the gate does not inspect the content of the diagnostic LLM's output—it only checks whether the extracted role string is in the pre-approved set and whether it is a privilege escalation from the failing role.

This asymmetry has an architectural implication: *as LLMs become stronger instruction-followers, structural enforcement transitions from complementary to necessary.* Framework designers should not rely on content-level provenance signaling alone (template selection, untrusted framing) but should pair it with structural privilege invariants at the recovery sink. Content-level defenses remain valuable as defense-in-depth—they may catch attacks that the structural invariant does not cover (e.g., content-level trust-escalation via high-trust template re-injection, which is outside the scope of this paper but is addressed in the broader agent-security literature Debenedetti et al. (2025); Zhan et al. (2025))—but they should not be the sole layer of defense for privileged recovery actions.

6.4. Recommendations for Framework Developers

Based on our analysis, we recommend that developers of self-healing LLM agent frameworks:

1. **Audit recovery sinks for missing authorization.** Check whether the role-replacement / topology-mutation primitive performs an authorization check on the value it consumes, or whether it consumes the diagnostic LLM's recommendation verbatim. The CWE-862 criterion from §3.4 provides a replicable methodology: if the sink performs a privileged operation without verifying that the caller is authorized to request that operation, the sink is vulnerable.

2. **Centralize privilege re-verification.** Do not scatter permission checks across individual recovery strategies. Centralize them at the orchestrator level, where the invariant "no recovery action exceeds the privilege of the failed execution" can be verified by inspecting one method. This locality is itself an OS principle: classical kernels enforce privilege invariants at well-defined boundaries (syscall entry, exception dispatch), not inside each driver or subsystem.

3. **Defer side effects, do not execute inline.** Side-effecting recovery strategies should not execute their side effects inline at strategy-return time. Instead, they should package the side effect as a deferred callback, set a `requiresReauthorization` flag on the result, and let the orchestrator execute the callback only after the gate has verified the privilege invariant. This turns the security property from an emergent property of independently-authored strategies into a local, inspectable property of one orchestrator method.

4. **Pair content-level with structural defenses.** Do not rely solely on prompt-level untrusted framing or instruction-hierarchy enforcement for privileged recovery actions. As models become stronger instruction-

followers, content-level defenses degrade; structural privilege invariants provide a model-independent backstop.

6.5. Migration Guide for Framework Developers

We now provide a step-by-step migration guide for porting the REAUTHORIZATION GATE design pattern onto an existing self-healing pipeline. The guide assumes a typical framework architecture with (i) a tool-execution layer that raises exceptions, (ii) a recovery orchestrator that catches failures and dispatches recovery strategies, and (iii) a permission checker that gates forward-path tool calls. The migration is designed to be incremental and backward-compatible.

Step 1: Add a `requiresReauthorization` flag to the recovery result type.. The first and lowest-risk step is to extend the recovery result type with a boolean flag `requiresReauthorization` (default `false`). No gate is added yet; this step only prepares the plumbing. Existing strategies that do not set the flag default to `false` and behave as before (no gate interception), so a framework can migrate strategies incrementally without breaking existing functionality.

Step 2: Defer side effects in side-effecting strategies.. For each side-effecting recovery strategy (role replacement, tool execution, model switching), refactor the strategy to package its side effect as a *deferred callback* stored in the recovery context metadata, rather than executing it inline at strategy-return time. The strategy sets `requiresReauthorization=true` and returns; the side effect is held in suspension. This step does not change observable behavior yet (no gate intercepts the flag), but it prepares the strategy for gate interception.

Step 3: Install the REAUTHORIZATION GATE at the orchestrator.. Modify the recovery orchestrator's main loop to intercept every result with `requiresReauthorization=true` *before* executing the deferred callback. The gate calls the existing `PermissionChecker` (the same one used on the forward path) to verify the privilege invariant: for role replacement, the new role must be in the pre-approved downgrade set of the original role; for tool execution, the requested tool must be in the original role's allowed set. If the check fails, the side effect is blocked and the orchestrator returns a failed recovery result; if it passes, the deferred callback executes. The gate is a single method that can be inspected to verify the invariant "no recovery action exceeds the privilege of the failed execution."

Step 4: Add a role-existence whitelist.. In addition to the privilege check, add a whitelist check: the proposed role must exist in the framework's role blueprint registry, not be an arbitrary LLM-suggested string. This closes the CWE-862 sink completely: even if the diagnostic LLM is tricked into suggesting an out-of-set role (e.g., `admin`), the whitelist rejects it before the side effect executes.

Step 5: Audit existing strategies for compliance.. After installing the gate, audit each existing recovery strategy to ensure that (a) every side-effecting strategy sets `requiresReauthorization=true`, and (b) no strategy executes side effects inline (bypassing the gate). This audit is local: it checks one flag per strategy, not the strategy's internal logic. Strategies that are pure diagnostic (no side effects) need no change. New strategies added after the migration inherit protection automatically by setting the flag.

Expected migration cost.. In the Recova testbed, the full migration touched 14 strategy classes and one orchestrator method, with sub-millisecond runtime overhead and zero false alarms on benign recovery (§5). The migration is designed to be backward-compatible: a strategy that does not set `requiresReauthorization` defaults to `false` and behaves as before (no gate interception), so a framework can migrate strategies incrementally without breaking existing functionality. The pattern does not require any modification to the forward execution path, the LLM, or the role/permission model.

7. Conclusion

Summary.. This paper introduced the *topology-mutation privilege-escalation* attack vector, a previously overlooked class of authorization failures on the recovery path of self-healing LLM agent systems. We showed that the recovery pipeline—designed to improve robustness by allowing a diagnostic LLM to propose remedial actions—can be abused as a *privilege-escalation trampoline*: a recovery strategy that proposes to mutate the agent topology (e.g., to replace a role or reconfigure a sub-agent) bypasses the authorization checks that gate the forward execution path, because the recovery machinery was never designed to re-verify privileges at the side-effect boundary. The contribution is architectural: we formalize the anti-pattern as a manifestation of CWE-862 (Missing Authorization) on the recovery sink, trace its root cause to the classical *confused deputy* problem, and propose a single OS-principled design pattern—the Reauthorization Gate—that ports the kernel exception-handler privilege invariant to the LLM agent recovery path.

Architectural contributions.. The paper makes three architectural contributions. First, we identify and formalize the *unchecked recovery sink* anti-pattern, in which a recovery strategy executes a privileged side effect (role replacement, topology mutation) without re-checking the authorization that gates the forward path; we trace its root cause to the implicit "failure = benign, retry is safe" assumption inherited from classical OS exception handling—an assumption that holds when failure payloads originate from the trusted computing base but breaks down when a diagnostic LLM proposes privileged remedial actions. Second, we propose the Reauthorization Gate, a single design pattern that enforces the Privilege-Downgrade Invariant ($P(\text{recovery action}) \leq P(\text{failed execution})$) at a centralized choke point through which all side-effecting recovery actions must flow; the pattern ports the OS principle that exception handlers never escalate privileges and the capability-based security tradition to the agent recovery path. Third, we show that the

pattern is framework-agnostic: it depends only on the existence of a recovery pipeline with declarable side effects and a permission checker, not on any specific agent architecture, making it portable to any self-healing agent system that exposes a topology-mutation or role-replacement primitive.

Validation summary.. A concise validation on the Recova testbed confirms that the REAUTHORIZATION GATE is implementable, effective, and imposes negligible cost on legitimate self-healing. The defended V2 attack success rate is **0.00 by construction**: the whitelist invariant rejects out-of-set roles deterministically, illustrating the key architectural principle that structural invariants enforced at a centralized choke point are LLM-independent and therefore robust to model scaling. The defense imposes zero false alarms on legitimate recovery and topology-mutation workloads, and sub-millisecond machinery overhead. Because the guarantee is structural rather than probabilistic, it does not degrade as instruction-following models become stronger, and it is not bypassable by paraphrased payloads or defense-aware adversaries—the gate inspects the proposed role against an authorization set, not the language of the diagnostic recommendation.

Empirical grounding.. The empirical evidence in this paper comprises a source-level structural analysis of the Recova recovery pipeline that identifies the `parseAndValidate(validate=false)` call in `TopologyMutationStrategy` as a CWE-862 missing-authorization site, and a measured end-to-end attack that achieves ASR 1.00 in the baseline configuration and ASR 0.00 in the defended configuration across $N = 30$ trials. The structural argument establishes that the recovery-path authorization boundary is independent of forward-path input defenses: even with forward-path input sanitizers, instruction hierarchies, or capability-based tool invocation fully enabled, the attack would still succeed because the privileged side effect is invoked by the recovery orchestrator, which forward-path defenses never inspect. This confirms that topology-mutation privilege escalation is a distinct authorization surface on the recovery path, not a variant of indirect prompt injection.

Limitations and future work.. The REAUTHORIZATION GATE enforces a *static* privilege invariant: it checks that the proposed role is in the legitimate downgrade set, but does not inspect *why* the recovery strategy proposed it. A sufficiently sophisticated payload could induce a strategy to propose an action that is individually permissible yet collectively harmful (e.g., a sequence of individually-legitimate downgrades that compose into an unintended capability). As future work, we propose *intent auditing via inner-monologue inspection*: instrumenting the diagnostic LLM's chain-of-thought to verify that the proposed recovery action is justified by the failure's actual cause, rather than by injected reasoning. This would add a semantic, model-level defense layer atop the syntactic, OS-level invariant introduced here—complementary rather than a replacement, and aligned with the broader trend of using LLMs to audit LLM behavior.

A second limitation concerns the scope of the validation: the testbed validation establishes implementability, efficacy, and cost, but broader empirical claims—cross-model robustness of the content-level diagnostics that feed the gate, additional production-framework topology-mutation primitives, and fully adaptive multi-

round adversaries that observe defense rejections and iteratively rewrite diagnostic recommendations—are not established in this architectural paper and are immediate future work.

8. Ethics Statement and Design Recommendations

8.1. System Identity and Authorship

The system studied in this paper, Recova, is a multi-agent LLM operating system developed and maintained by the authors of this paper. The attack vector studied (V2: `TopologyMutationStrategy` privilege escalation via missing authorization on the role-replacement primitive) was identified during a security review of the authors' own codebase, not through adversarial probing of a third-party system. We are the maintainers of Recova; the work is therefore a *self-study of an authorization flaw in our own prototype*, not a black-box security audit of an external deployed system.

Positioning.. We explicitly avoid the term "vulnerability disclosure" for this work. "Disclosure" conventionally refers to reporting a flaw discovered in a third-party, deployed system to its maintainers. In our setting, the system is the authors' own prototype, has never been publicly released, and has no external users. The contribution is therefore better described as: *(i) we identify and characterize an authorization attack vector on the recovery sink of self-healing LLM agents (topology-mutation privilege escalation), (ii) we instantiate and study it on a representative self-healing architecture prototype developed by the authors, and (iii) we propose and validate a portable design pattern (*Reauthorization Gate*) that enforces the privilege-downgrade invariant.* We frame the evidence accordingly: the validation results are measurements on the authors' own prototype under a specific configuration, not field observations from a deployed system.

Naming.. The system under study is referred to throughout this paper by the placeholder Recova (the LaTeX macro `\system{}` expands to the chosen name). The name used here is a paper-level label for the prototype under study; it is not a product name and does not refer to any externally deployed system.

8.2. Internal Remediation Timeline

Because the attack vector is in the authors' own system, no external CVE request or third-party notification was required. The internal review and remediation timeline is:

- **2026-04-19:** V2 identified during a red-team review of the recovery pipeline. Root cause identified as the `parseAndValidate(validate=false)` call in `TopologyMutationStrategy`, which parses a diagnostic-LLM-proposed role without checking it against the privilege set authorized for the failed execution.
- **2026-04-26:** Defense design finalized (unified Reauthorization Gate enforcing the Privilege-Downgrade Invariant). Implementation completed in a feature branch.

- **2026-05-10:** Defense merged to the main branch and deployed to the internal staging environment. No production incident has been attributed to V2 prior to the fix; the issue was discovered proactively, not in response to an attack.
- **2026-05-17:** Validation (reported in §5) started on the patched codebase; the baseline measurements were obtained by toggling the defense off via a runtime flag on the same patched code, ensuring the only difference between configurations is the defense itself.
- **2026-06-21:** Paper submitted for double-blind review. The patched code is the current code; no separate "patched release" was issued because the system has not been publicly released.

8.3. Ethics Statement

No third-party harm.. Because Recova is the authors' own system and has not been publicly released, the experiments in this paper do not expose any third-party user to risk. No live user data was used in the validation; all attack payloads were synthetic and directed at canary tools deployed in an isolated test environment.

Attack payload release.. We commit to releasing the attack payloads, the validation harness, and the anonymized real-LLM response logs upon publication. We follow the principle of *responsible release*: the released artifacts include only payloads that target *structural* authorization properties of the recovery sink (the proposed role and its authorization set) and do not include model-specific jailbreaks that would transfer to unrelated production systems. The defense is shipped alongside the attack payloads, so that any party reproducing the attack also has the fix available.

LLM API usage.. The validation consumed a small number of paid API calls (approximately 90 trials across the V2 attack, benign recovery, and benign topology-mutation configurations, $N = 30$ per configuration) to DeepSeek-V4-Flash (via SenseNova) and Qwen2.5-72B-Instruct (via SiliconFlow) for the Recova testbed validation reported in §5, at a total cost of under US$5. No model provider's terms of service were violated; the prompts used are research-purpose red-team prompts targeting our own system, not attempts to extract training data or bypass model safety training in general.

Communication to other framework maintainers.. Our evidence for cross-framework impact is structural rather than exploit-based, and we calibrate our communication accordingly. The Reauthorization Gate pattern applies, in principle, to any self-healing agent system that exposes a topology-mutation or role-replacement primitive on its recovery path. We frame our communication to other framework maintainers as a *design recommendation* ("if your recovery path can mutate the agent topology or replace a role, consider enforcing a privilege-downgrade invariant at a centralized reauthorization gate before the side effect executes"), not as a vulnerability report on any specific third-party system. We will communicate this design

recommendation, with a pointer to the pattern and reference implementation, to maintainers of widely-deployed agent frameworks at least 45 days prior to publication, and will provide the program committee with verification evidence (issue links, acknowledgment emails) upon request.

Broader impact.. Self-healing mechanisms are becoming a standard component of production LLM agent systems, and the recovery-path authorization boundary they introduce is currently under-recognized. By naming and characterizing this boundary, we aim to enable framework authors to design recovery pipelines with explicit privilege invariants from the outset, rather than retrofitting them after deployment. The cost of the proposed defense (sub-millisecond machinery overhead, zero false alarms) is low enough that we expect it to become a default component of future self-healing agent architectures that expose privileged topology-mutation primitives.

References

Abdelnabi, S., Greshake, K., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you've asked for: Exploiting tool-integrated LLM agents via indirect prompt injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec).

[Authors], 2026. Recova: Self-healing multi-agent operating system. The system under study in this paper; see §8 for authorship and design-recommendation status. Cited as the source of the multi-tier recovery architecture analyzed in this work.

Bovet, D.P., Cesati, M., 2005. Understanding the Linux Kernel. 3rd ed., O'Reilly Media.

Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., Tramèr, F., 2025. Defeating prompt injections by design. arXiv preprint arXiv:2503.18813 Proposes CaMeL, a capability-based defense that extracts control and data flows from a trusted query and enforces security policies when tools are called, preventing untrusted data from impacting program flow on the forward execution path.

Fabry, R.S., 1974. Capability-based addressing, in: Communications of the ACM, pp. 403–412.

Gou, Z., Shao, Z., Gan, Y., Akihara, J., Qiu, X., Li, Y., Liu, P., Matsuo, Y., 2024. CRITIC: Large language models can self-correct with tool-interactive critiquing, in: International Conference on Learning Representations (ICLR).

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. arXiv preprint arXiv:2302.12173 .

Hardy, N., 1988. The confused deputy (or why capabilities might have been invented). ACM SIGOPS Operating Systems Review 22, 36–38. doi:10.1145/54289.871709. originated the "confused deputy" problem: a privileged program tricked by a less-privileged caller into misusing its authority. Direct analog of V2's privilege-escalation trampoline.

Hevner, A.R., March, S.T., Park, J., Ram, S., 2004. Design science in information systems research. MIS Quarterly 28, 75–105. doi:10.2307/25148625.

Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., Kiciman, E., 2024. Defending against indirect prompt injection attacks with spotlighting, in: arXiv preprint arXiv:2403.14720.

Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S.K.S., Lin, Z., et al., 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv preprint arXiv:2308.00352.

Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C., Loingtier, J.M., Irwin, J., 1997. Aspect-oriented programming, in: Proceedings of the 11th European Conference on Object-Oriented Programming (ECOOP), pp. 220–242. doi:10.1007/BFb0053381.

LangChain, 2024. LangGraph: Building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph.

Linux Kernel Documentation, 2024. The seccomp Linux kernel subsystem. https://www.kernel.org/doc/html/latest/userspace-api/seccomp_filter.html.

Liu, Y., Jia, Y., Geng, R., Jia, J., Gong, N.Z., 2024. Formalizing and benchmarking prompt injection attacks and defenses, in: USENIX Security Symposium. ArXiv:2310.12815.

Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P., 2023. Self-refine: Iterative refinement with self-feedback, in: Advances in Neural Information Processing Systems (NeurIPS).

MITRE Corporation, 2024. CWE-862: Missing authorization, in: Common Weakness Enumeration. The CWE class corresponding to V2's missing permission re-check on the role-replacement primitive.

OWASP, 2025. OWASP top 10 for LLM applications (2025 edition). https://genai.owasp.org/llmrisk/. Industry-standard taxonomy of LLM application risks. *LLM01: Prompt Injection* covers forward-path injection; the recovery-channel surface studied in this paper is not enumerated.

Reynolds, P., Killian, C., Vahdat, A., Vahdat, A., 2000. Capsules: A compound document architecture for OS-level sandboxing, in: Proceedings of the 4th USENIX Windows Systems Symposium.

Saltzer, J.H., Schroeder, M.D., 1975. The protection of information in computer systems. Proceedings of the IEEE 63, 1278–1308.

Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., Yao, S., 2023. Reflexion: Language agents with verbal reinforcement learning, in: Advances in Neural Information Processing Systems (NeurIPS).

Silberschatz, A., Galvin, P.B., Gagne, G., 2003. Operating System Concepts. 7th ed., Wiley.

Wallace, E., Stern, M., Song, D., 2024. The instruction hierarchy: Training LLMs to prioritize privileged instructions, in: Proceedings of the 41st International Conference on Machine Learning (ICML).

Willison, S., 2023. Prompt injection: What's the worst that can happen? https://simonwillison.net/2023/Oct/14/prompt-injection/.

Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., et al., 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. arXiv preprint arXiv:2308.08155.

Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O., 2024. SWE-agent: Agent-computer interfaces enable automated software engineering. arXiv preprint arXiv:2405.15793.

Zhan, Q., Fang, R., Panchal, H.S., Kang, D., 2025. Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents, in: Findings of the Association for Computational Linguistics: NAACL 2025. Evaluates 8 defenses against indirect prompt injection and bypasses all of them with adaptive attacks, achieving >50% ASR. The attack targets the forward execution path (tool outputs), not the recovery channel.

Zou, W., Geng, R., Wang, B., Jia, J., 2024. Poisonedrag: Knowledge corruption attacks to retrieval-augmented generation of large language models. arXiv preprint arXiv:2402.07867.